Star Light, Star Bright: What Size of Star May I Classify Tonight?

Siera Edwards, Haydn Duncan, Mamadou Dem, Bradley Allen

Department of Data Science, Willamette University

DATA 252: Models and Machine Learning

Dr. Hank Ibser

May 5, 2025

Introduction

It is said that without stars, life would not exist on Earth (Bates, 2021). Humans have used stars to tell time, predict seasons, and navigate the seas (Mattson et al., 2013). Because stars have played an instrumental role in human civilization, recent research has focused on identifying and classifying stars using modern technology. Stars are composed of various burning gases packed densely together, creating a variety of colors, sizes, and brightnesses (*Star*, 2025). Today, researchers determine a star's stellar classification by its absolute magnitude (intrinsic brightness), temperature (as determined by color), and radius (The Editors of Encyclopaedia Britannica, 2025). Our project leverages machine learning models to predict star size using similar predictors (e.g., absolute magnitude, temperature). Models used in this project include k-Nearest Neighbors, Logistic Regression, Simple Linear Regression, and many more. The goal was to find the best classification model given limited variables selected for optimal noise level considering our data was "too accurate" when all variables were included.

The data used for this project was the *Star Dataset to Predict Star Types* available from Kaggle (Baidya, 2019). The data set included 7 variables and 240 observations. The variables were numeric (5) and strings (2), and included temperature (in kelvin), luminosity, radius, absolute magnitude, star type (e.g., Brown Dwarf, Red Dwarf, etc.), color, and spectral class (e.g., O, B, A, F, G, K, M). Data usability was rated at 10 out of 10. A scatter plot of the data sorted by absolute magnitude and temperature with fill set to star type revealed the data were clustered (Figure 1). This clustering resulted in highly accurate prediction models. To properly test the prediction models, some variables were removed throughout the exploration process, generating various degrees of inaccuracy.

Method

kNN Classification

This model was created using a 70/30 split of training/testing data. Temperature (K), luminosity (brightness), and star color (e.g., yellowish, white, red, orange, blue-white, blue), were used to predict star type. As can be seen in Figure 2, these variables provided adequate overlap between star type grouped by color, such that the initial model was about 61% accurate in predicting star type given a randomly selected *k*-value of 7 (Figure 4A). To find the optimal value for *k*, we employed a *for loop* to run 15 iterations (e.g., $k = 1{:}15$) of kNN modeling using the same variables. As can be seen in Figure 3, the error rate for each value of *k* was plotted. In the given range, $k = 3$ had the lowest error value of about 0.31. Using $k = 3$, the model accuracy increased to 69.4%, about eight percent more than the original model (Figure 4B). Given the restricted set of variables, 51% was an acceptable accuracy value. For example, using absolute magnitude and radius produced models with 100% accuracy regardless of the *k*-value.

Classification Tree

For the Tree Model, we construct and evaluate a Classification Tree to predict star size (small or large) from features like temperature and star color. Classification trees are widely used because they are easy to use and interpret, especially when trying to understand how features influence predictions.

The dataset was split into a 70/30 training/testing split using the caret package in R. We used a modification of the star type variable for this model which was a binary form of the variable which will be used throughout this project. To create this new variable, we divided our star type column into 2 components, 0 for dwarf planets (star types 0, 1, and 2) and 1 for main sequence and giant stars (star types 4, 5, and 6) which made up our new response variable: star

size. We trained the model using the rpart() function with planetSize as the target variable. As can be seen in Figure 5, our first decision tree split was at Temperature < 3560K between cooler and warmer stars. For more warm stars, the next split was based on whether the Star.color was White. Every terminal leaf node classified a star as dwarf (class 0) or larger star (class 1) with the corresponding probabilities.

The first model scored 68.1% accuracy, as it can be observed from the confusion matrix (Figure 6). The tree correctly classified 21 small stars and 28 big stars, while it incorrectly classified 15 big stars and 8 small stars (Figure 6). This was already very good considering that the tree utilized only two input variables. To avoid overfitting, we verified model complexity using a complexity parameter (CP) plot (Figure 7). The best CP value that minimized cross-validation error was found to be 0.01. We then pruned the tree similarly using prune(), but to our dismay, the pruned tree ended up with the same identical structure as the original tree (Figure 8). This shows that the original tree was already optimally sized and no further pruning was needed.

One of the biggest advantages of employing a decision tree in this project was its readability. While more sophisticated models such as random forests or boosting might yield more accurate predictions, they tend to lose the distinct "if-then" reasoning that decision trees provide. In an environment such as stellar classification, where it is helpful to know the reasons behind a prediction, this readability is a plus.

Overall, the classification tree achieved a good balance between simplicity and performance, which makes it a good model for understanding how star color and temperature affect a star's classification.

Bagging Model

The next model that we created, a bagging model, was to predict the star type (0-5) and is a different form of a tree model. Bagging stands for "Bootstrap Aggregation." This method takes random bootstrap samples of the data to make trees. It then averages the set of observations to reduce variability, however, we also lose interpretability with this method. To predict star type, we gave the model luminosity (Lo), temperature (K), and star color. After we built the model on the training dataset (70%), we tested it on the test dataset (30%) to see how it would perform. When we did this, it had an accuracy of 83.3% and made the most mistakes on star types 0 and 5. It was able to accurately predict star type 2. While this model has a high rate of accuracy, bagging can have certain problems. Sometimes the trees can be relatively similar and the model will be dominated by a few strong predictors. Additionally, bagged trees can be highly correlated and may not lead to a large reduction in variance when averaging. Thus, in general, this may not be the most effective model for predicting but it has a high rate of accuracy and can predict more sizes of stars in our situation.

Logistic Regression

Another model we used to predict star size (dwarf vs larger star) was logistic regression. This model uses probabilities to determine how to classify each data point. We began with a simple logistic regression where we used temperature (K) to predict the star size, shown in Figure 9. This model used a training set of 70% of the data to learn patterns and we tested it on the test dataset containing the other 30% of the data. This model had an accuracy of 54.2% with 11 false dwarf predictions and 22 false main sequence or larger stars, .

Then we decided to try multiple logistic regression, beginning with backward variable selection. This model starts with all possible explanatory variables then removes them

individually until it has reached the lowest possible AIC. This value is the Akaike Information Criterion and is used to measure how well a model fits the data and tries to prevent overfitting the data. At this point, our explanatory variables included only temperature, luminosity, and star color. When backwards selection was used, it determined that the most effective model was one that used only luminosity (Lo) to predict the star size. The model first removed star color, then temperature for a change of AIC from 16.01 to 7.69. This value helps to compare between models. Next we tried forward variable selection to see if it came to the same conclusion as the backward selection. This method starts with no variables and only adds new ones if they bring down the AIC. This model began with an AIC of 234.9 and brought it down to 7.69 by adding only luminosity. It determined that having only this variable created the best fit for the data. This showed that both methods of variable selection agreed that luminosity would be the best predictor for star size using logistic regression. We then graphed that model on our data to see what the fit looked like, shown in Figure 10.

Since both of these models use the same predicting variable, they have the same AIC value. They also have the same accuracy at 100%. This is the most accurate model we can get after we removed absolute magnitude and radius from the data to purposefully add some noise. The model using temperature is a good example of how logistic regression works using probabilities, but, given that we have a more accurate model for prediction, it should not be used. It is very interesting that the model determined that only using one variable was the most accurate but it makes more sense when we graph luminosity and see how the data points are organized and realize that predicting with just these values seems more simple and accurate.

LDA and QDA model

The goal of this model was to predict star size based on two explanatory variables, Luminosity and Temperature. Firstly, we plotted the distributions of both Luminosity(X-axis) and Temperature(Y-axis) on their respective axis to see what their distribution looks like. Looking at the plot, we can see that smaller stars seem to have a lower luminosity than larger stars.

Secondly, we plotted the LDA model in a partition plot to observe the boundary line and where all the 1's and 0's are congregating (Figure 11). As can be seen in Figure 12, the error rate listed at the top is 0.173 and most of the errors that are observed are located near the boundary line (which is to be expected). We calculated the calculation accuracy of the model and came out to be 0.827 (Figure 12). Following the LDA partition plot we made a QDA partition plot. This plot seemed to account for a lot of the errors along the boundary line and we're only left with a few errors really tight along the y-axis. This model's accuracy was greatly improved with an error rate of only 0.006 and a calculated classification accuracy of 0.994 (Figure 13).

Discussion

Conclusion

In conclusion, we performed two levels of prediction on this dataset. One was predicting the star type given the 6 initial types at the beginning. The other was a binary classification predicting whether a star was considered dwarf or main sequence/giants. The methods we used for the 6 types were KNN and a bagging model. The KNN model yielded results of an accuracy around 69%. Our bagging model did come out to be the best predictor with an accuracy of around 83.3%. For the binary classification, we used a tree model, logistic regression, QDA, and LDA models. The tree model used Temperature and star color to predict the star size, resulting in

an accuracy of 68%. Our logistic model made its prediction using the Luminosity feature and it came out with a resounding accuracy of 100%. Looking at the plot does bring up some concern regarding overfitting the model to the dataset. Then looking at the LDA and QDA models which were plotted both with Luminosity and temperature as its variables, we found the QDA model to be the most accurate with an accuracy score of 99.4%. The LDA model had the worst accuracy of the three with an accuracy of 82.7%. These models all help us better understand the patterns between star classification, temperature, and luminosity. This technology can help astronomers and data scientists automate the star classification process to reduce monotonous work and to allow more time for larger discoveries to be made.

Additional Models

Boosting

This model predicted star type using temperature (K), luminosity, and star color. The boosting model was calculated using a 70/30 training/testing split. The best model was found using 10 boosting iterations of 50 rounds (trees) per iteration with a learning rate, eta, or 0.4 (Figure 14). As can be seen in Figure 15, this resulted in a correct rate of 87.5%, with most errors off by one (e.g., type 4 classified as either a 3 or 5). This was an improvement over the default bagging model by about 4%. Over all, this model performed very well given the modified variables used.

Random Forest

This model predicted star size (dwarf or larger) using temperature (K) and star color. The random forests were created using a 70/30 training/testing split. The training model had an error rate of 20.24%, with the majority of errors predicting dwarfs as giants (22) vs predicting giants as dwarfs (12) (Figure 16). The best model was found using 10 boosting iterations of 50 rounds

(trees) per iteration, given two randomly selected predictors improved on this model with fewer errors predicting dwarfs and giants (9) vs giants as dwarf (15), resulting in a model accuracy of about 67% (Figure 17). Temperature (K) was the most commonly used variable (30%), followed by star color white (25%), and other colors in descending order (Figure 18). This model was slightly less accurate than our kNN model, making it the model with the lowest accuracy we explored.

Figure 1: Scatter Plot of Stars Data: Absolute Magnitude by Temperature

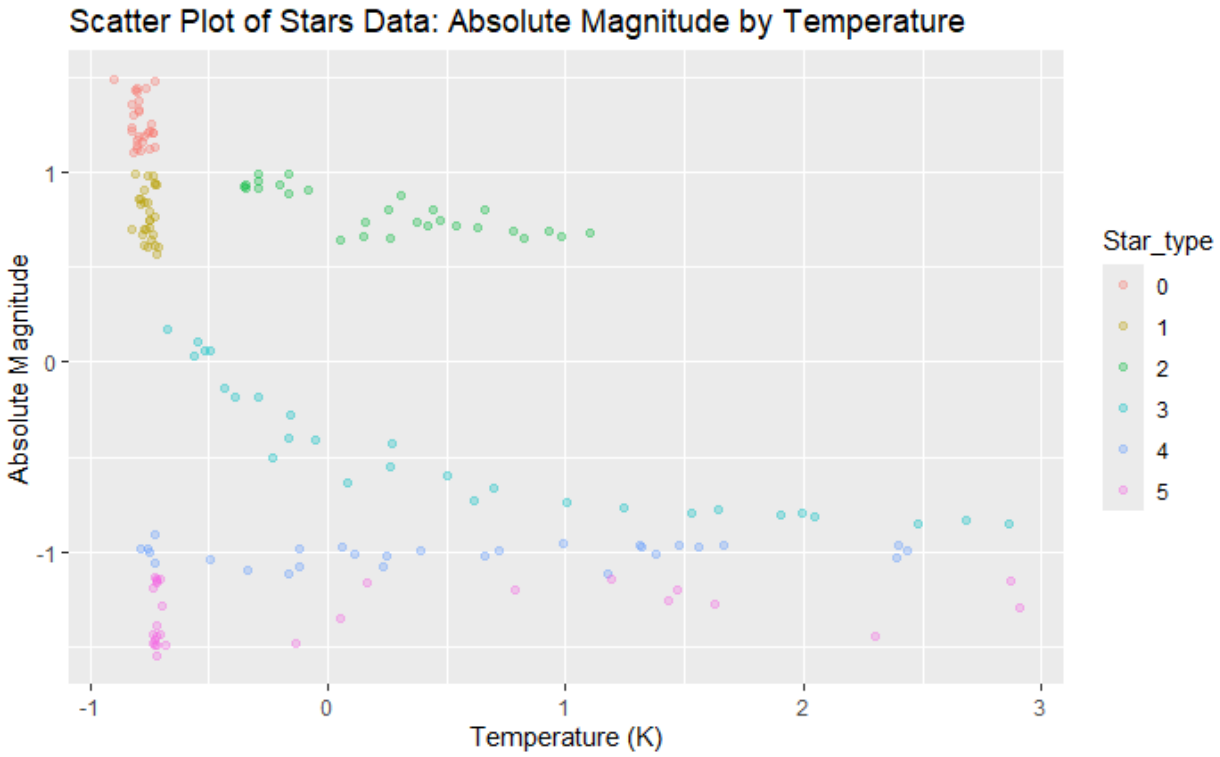


Figure 2: Star Color by Temperature Scatter Plot

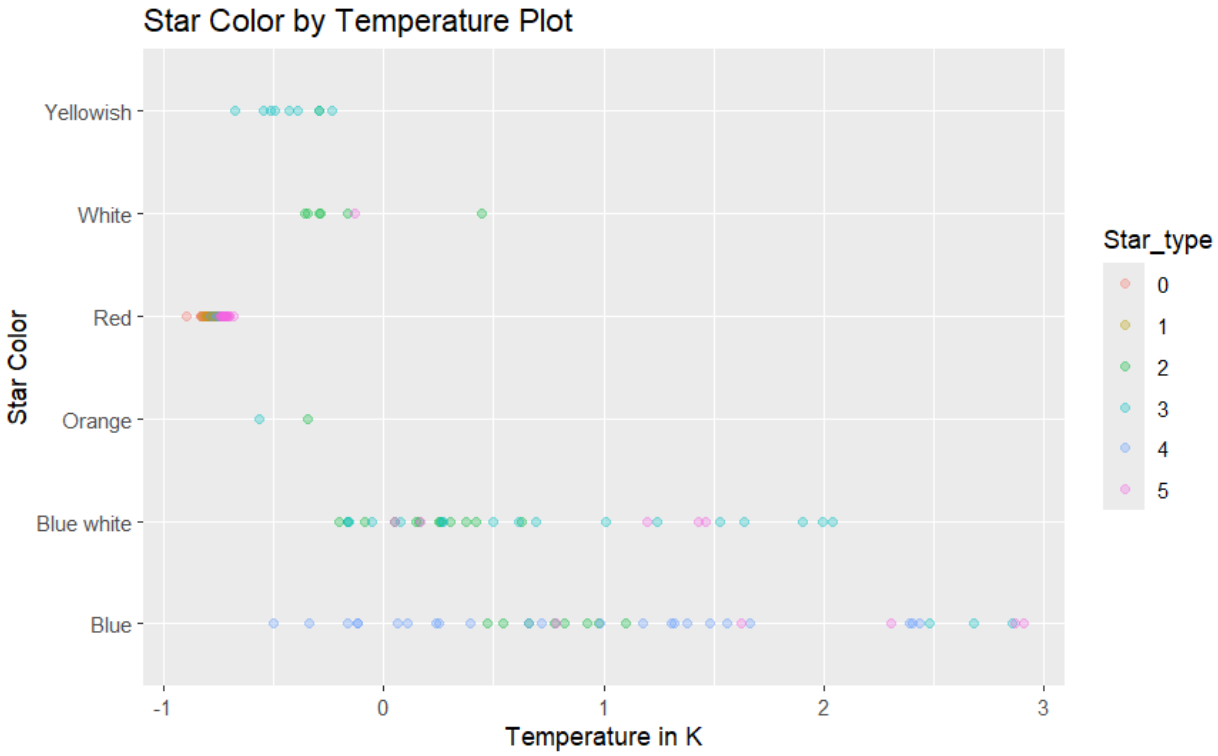


Figure 3: kNN Error Plot

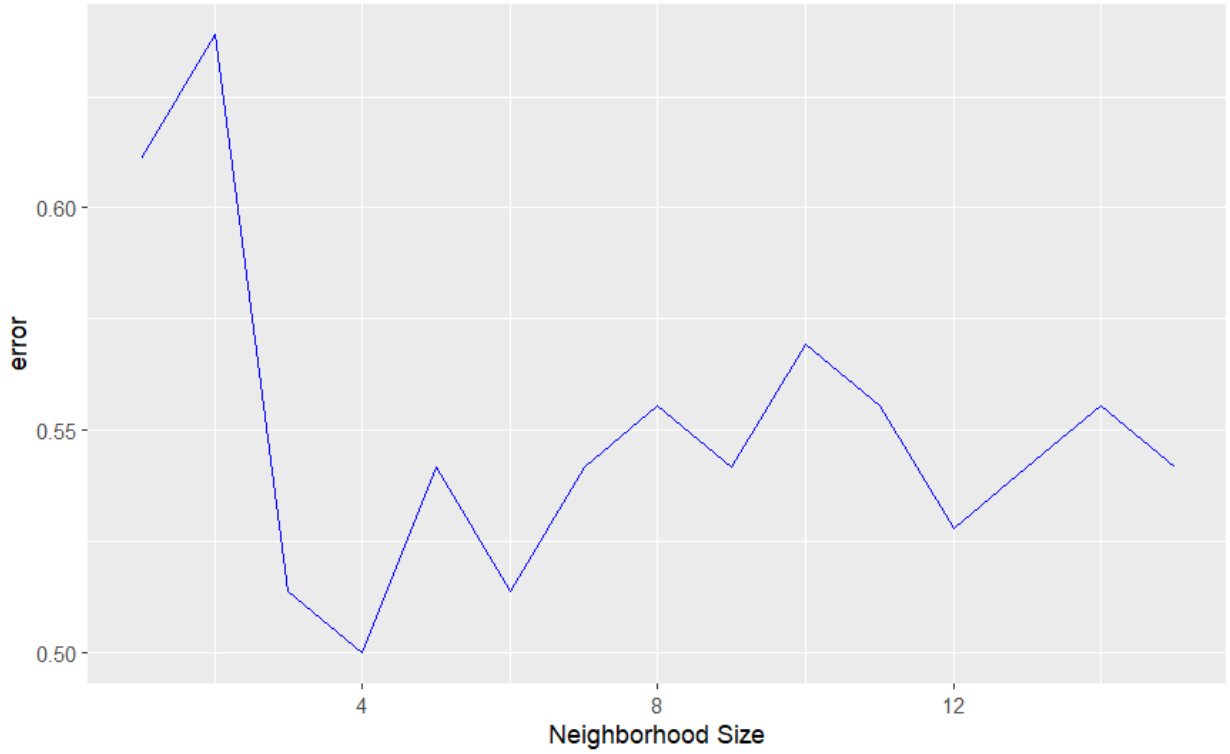
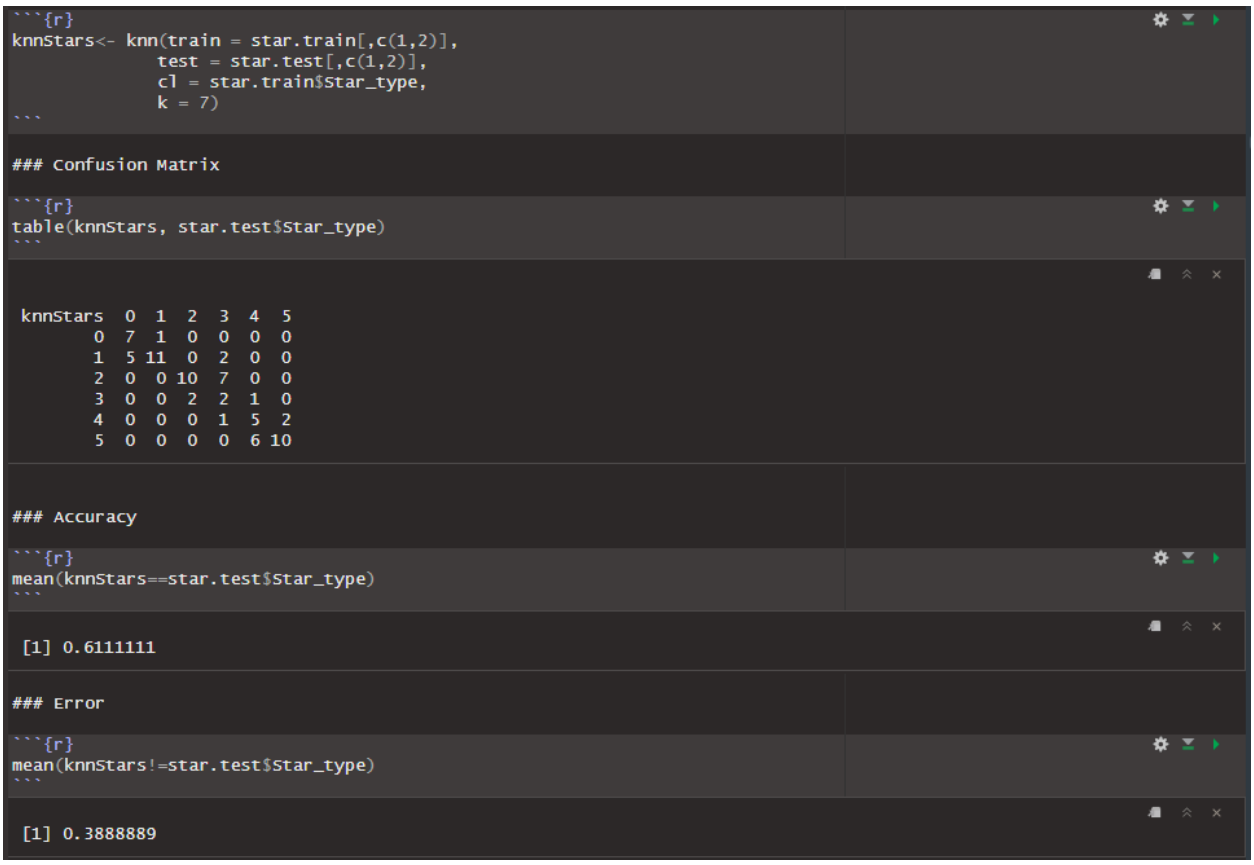


Figure 4A: kNN Confusion Matrix, Error Rate, and Prediction Rate for $k = 7$



```r
knnStars<- knn(train = star.train[,c(1,2)],
               test = star.test[,c(1,2)],
               cl = star.train$Star_type,
               k = 7)
```

Confusion Matrix

```r
table(knnStars, star.test$Star_type)
```

```
 knnStars  0  1  2  3  4  5
        0  7  1  0  0  0  0
        1  5 11  0  2  0  0
        2  0  0 10  7  0  0
        3  0  0  2  2  1  0
        4  0  0  0  1  5  2
        5  0  0  0  0  6 10
```

Accuracy

```r
mean(knnStars==star.test$Star_type)
```

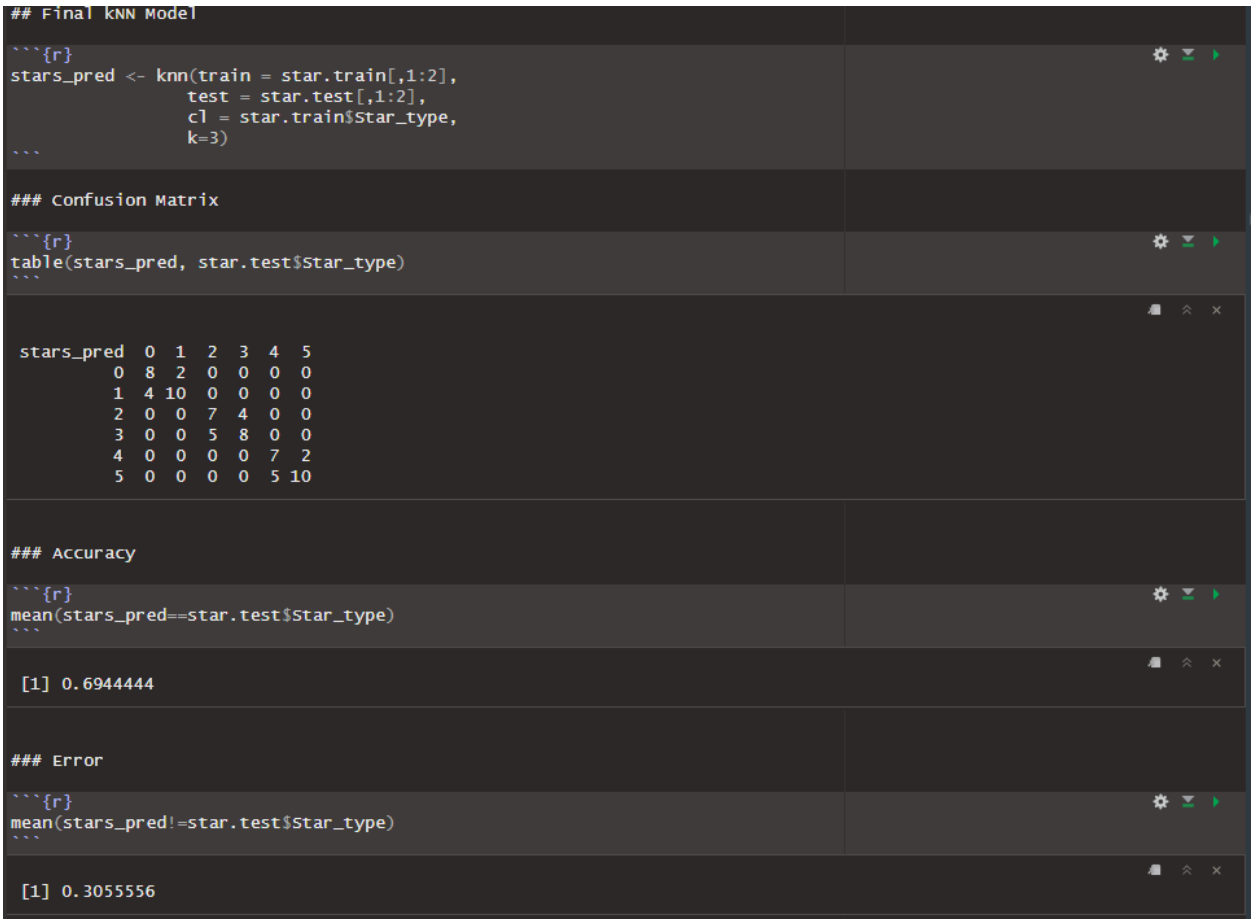
```
 [1] 0.6111111
```

Error

```r
mean(knnStars!=star.test$Star_type)
```

```
 [1] 0.3888889
```

Figure 4B: Optimal kNN Confusion Matrix, Error Rate, and Prediction Rate



Final kNN Model

```{r}
stars_pred <- knn(train = star.train[,1:2],
                  test = star.test[,1:2],
                  cl = star.train$Star_type,
                  k=3)
```

Confusion Matrix

```{r}
table(stars_pred, star.test$Star_type)
```

```
 stars_pred  0  1  2  3  4  5
          0  8  2  0  0  0  0
          1  4 10  0  0  0  0
          2  0  0  7  4  0  0
          3  0  0  5  8  0  0
          4  0  0  0  0  7  2
          5  0  0  0  0  5 10
```

Accuracy

```{r}
mean(stars_pred==star.test$Star_type)
```

```
 [1] 0.6944444
```

Error

```{r}
mean(stars_pred!=star.test$Star_type)
```

```
 [1] 0.3055556
```

Figure 5: Tree Model

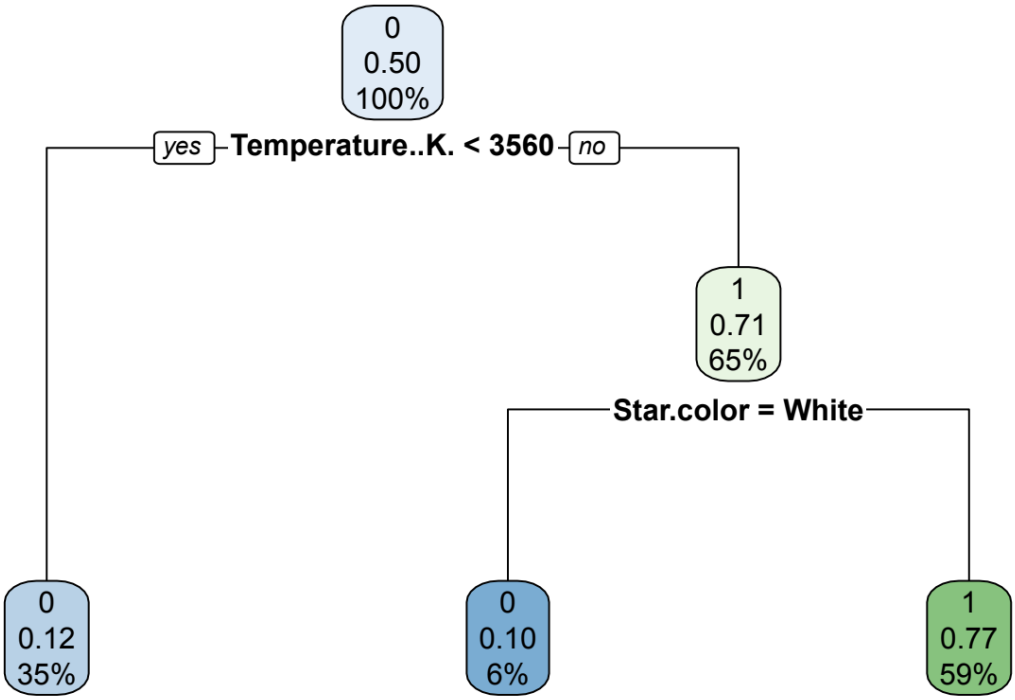


Figure 6: Confusion Table and Accuracy

Step 5: Predict

```
##     predTree1
##       0  1
##    0 21 15
##    1  8 28
```

Confusion Table

```
[1] 0.6805556
```

Model Accuracy

Figure 7: CP Plot

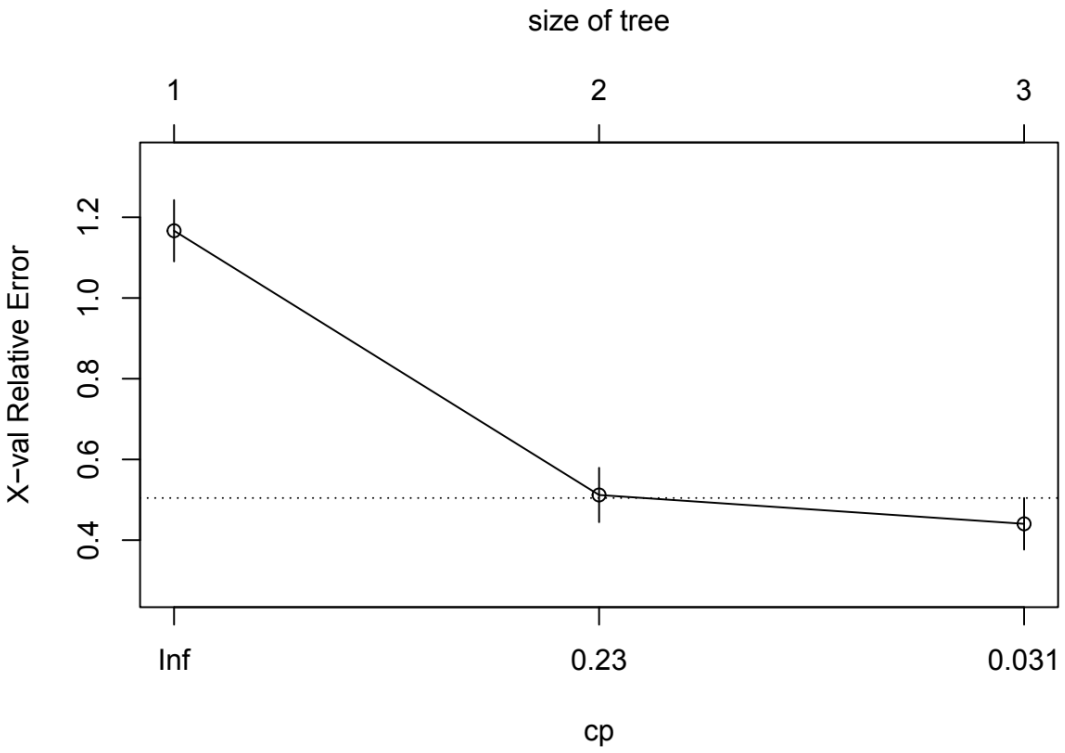


Figure 8: Pruned Tree Model

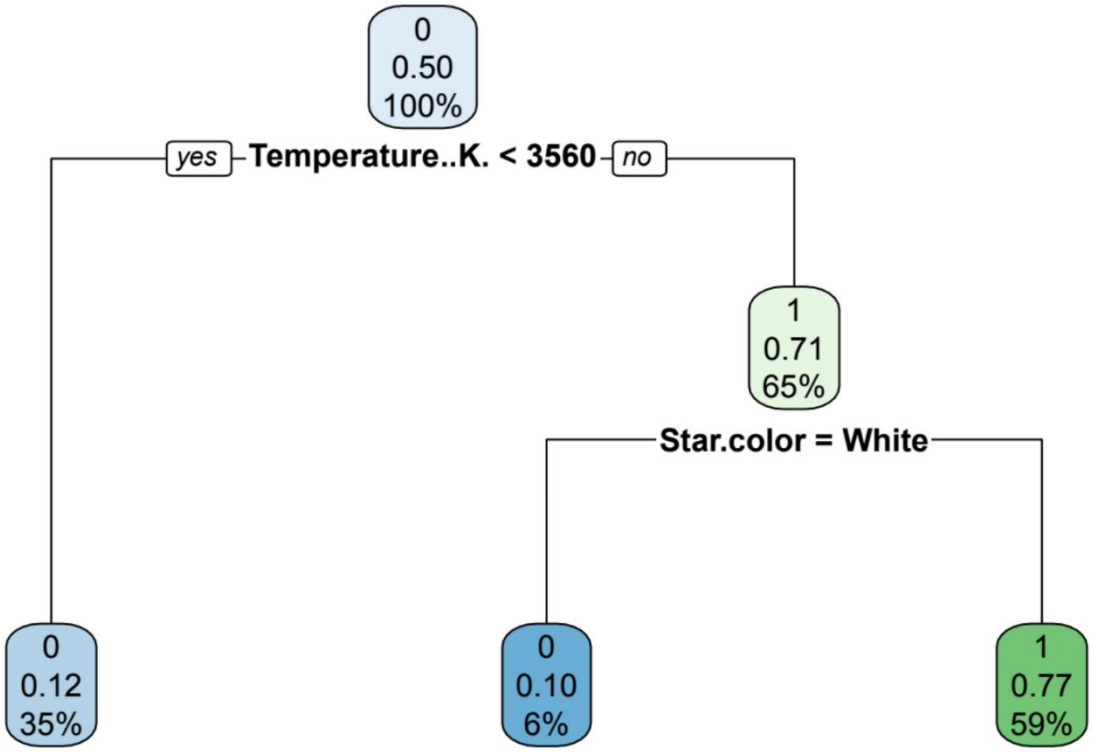


Figure 9: Simple logistic regression, using temperature (K) to predict star size.

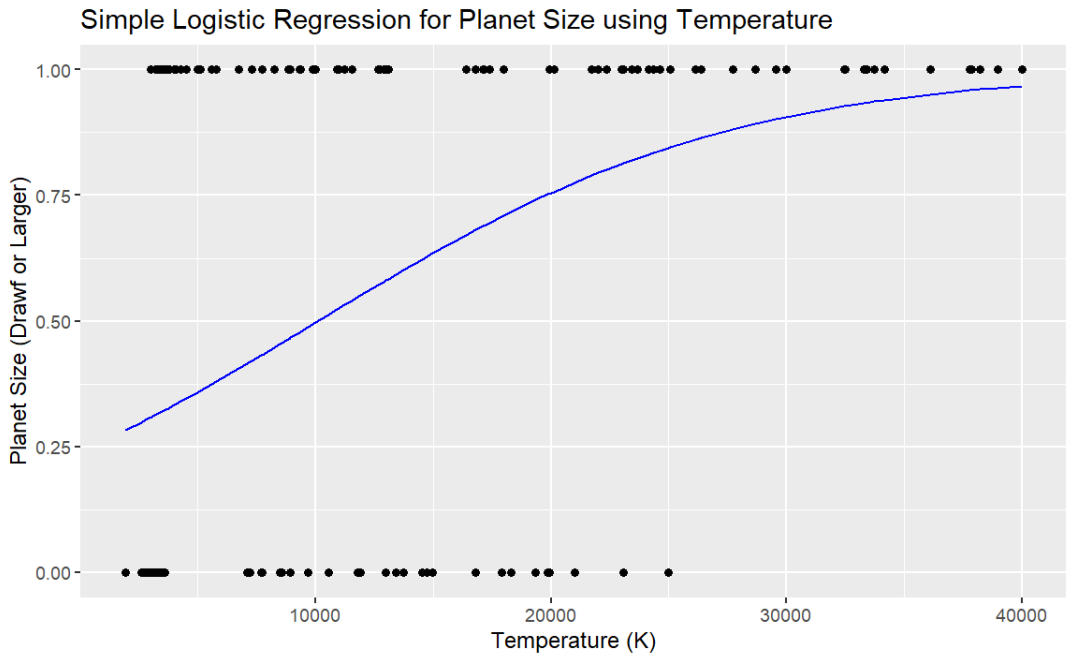


Figure 10: Best logistic regression model, using luminosity (Lo) to predict star size.

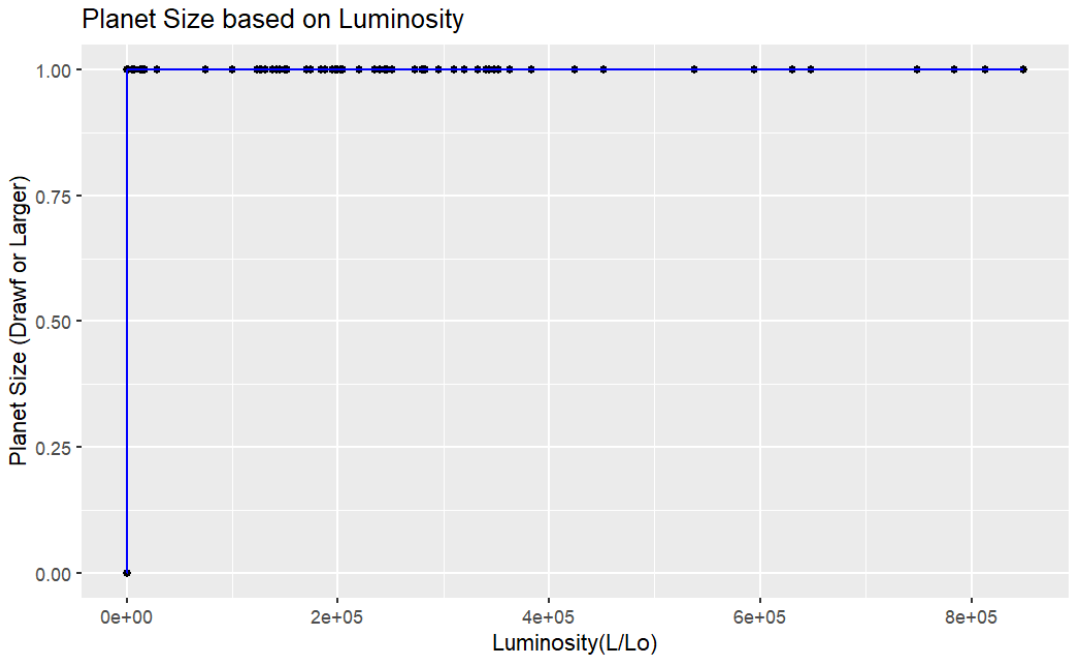


Figure 11: LDA Group Plot

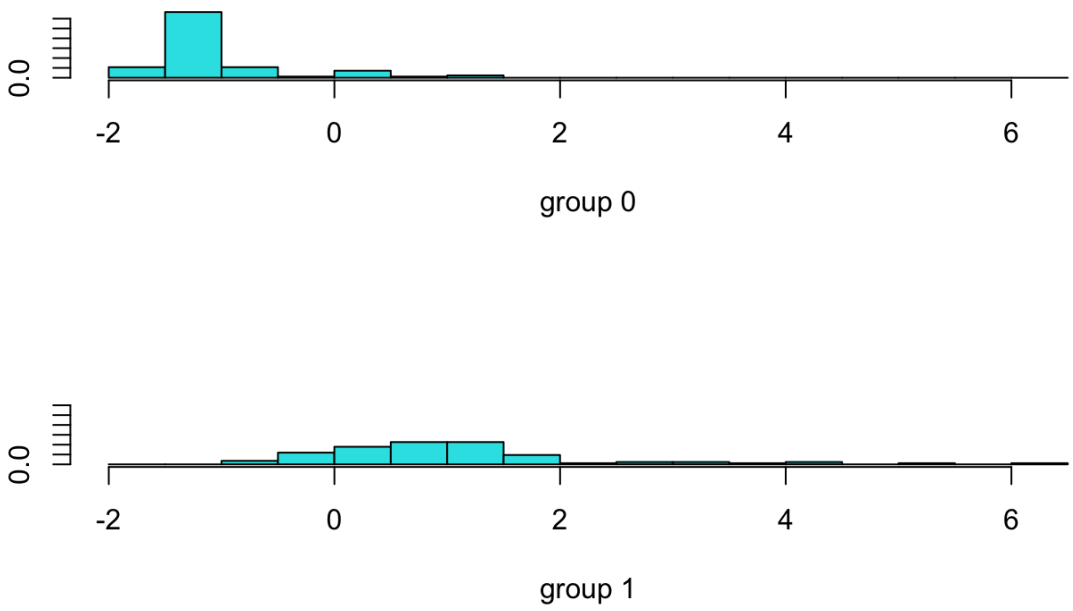
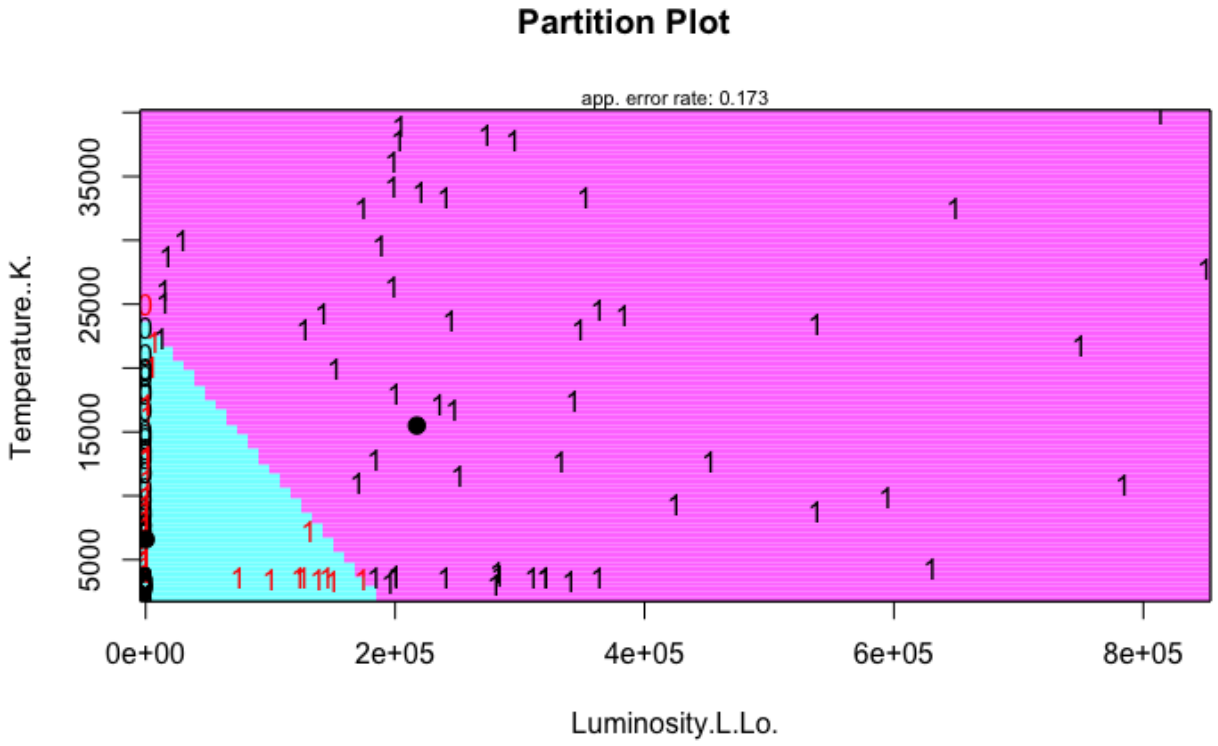


Figure 12: LDA Partition Plot and Accuracy

Partition Plot



Model Accuracy

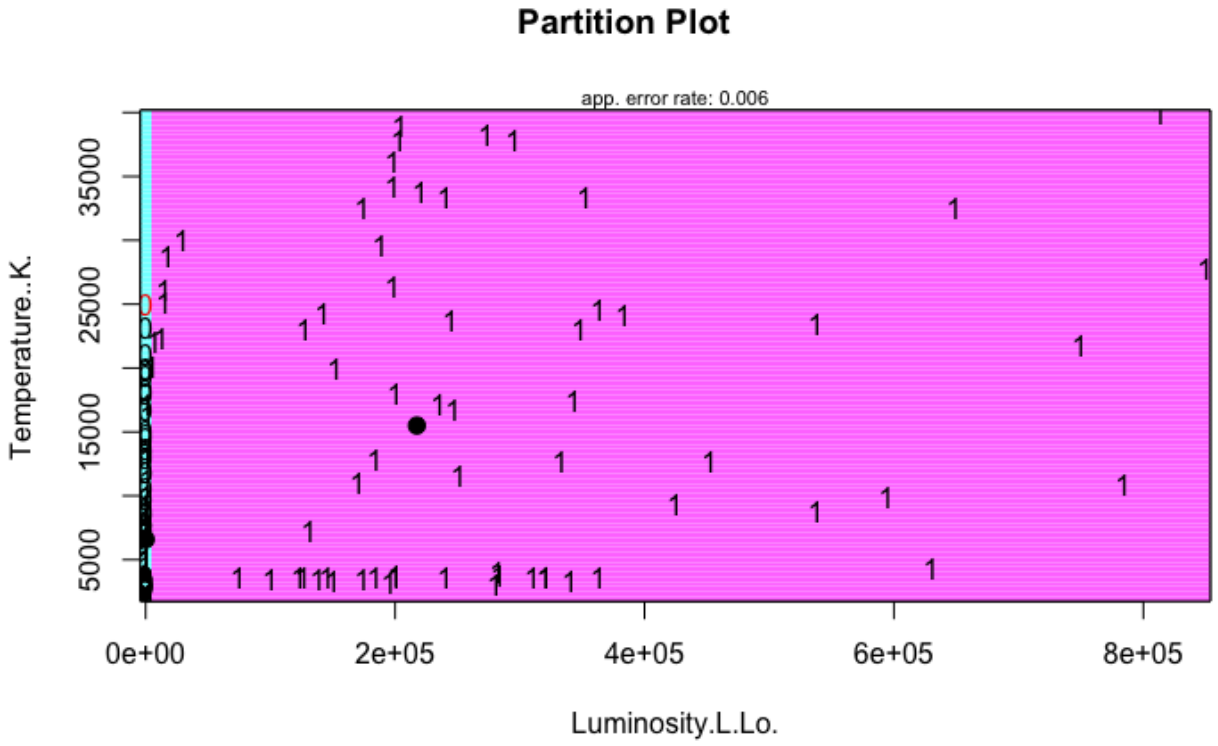
Correct Rate

```
mean(predLDA$class==da.testCaret$starSize)
```

```
## [1] 0.827381
```

Figure 13: QDA Partition Plot and Accuracy

Partition Plot



Model Accuracy

Correct Rate

```
mean(predQDA$class==da.testCaret$starSize)
```

```
## [1] 0.9940476
```

Figure 14: Boosting Model and Tuning Parameter Information

```
caretBoost <- train(Star_type ~.,
                    data = bag.trainCaret,
                    method="xgbTree",
                    trControl = trainControl("cv", number = 10),
                    verbosity = 0)
```

Best Tuning Parameter

```
##    nrounds max_depth eta gamma colsample_bytree min_child_weight subsample
## 61      50         1 0.4     0              0.6                1         1
```

Figure 15: Confusion Matrix and Correct Rate

Confusion Matrix

```
table(predCaretBoost, bag.testCaret$Star_type)
```

```
##
## predCaretBoost  0  1  2  3  4  5
##              0 11  1  0  0  0  0
##              1  1 11  0  0  0  0
##              2  0  0 12  0  0  0
##              3  0  0  0 12  0  0
##              4  0  0  0  0 10  5
##              5  0  0  0  0  2  7
```

Correct Rate

```
mean(predCaretBoost==bag.testCaret$Star_type)
```

```
## [1] 0.875
```

Figure 16: Random Forest Model and Tuning Parameter

```
set.seed(252)
caretRF <- train(starSize ~.,
                 data = log.trainCaret,
                 method = "rf",
                 trControl = trainControl("cv", number = 10),
                 importance = TRUE
)
```

Best tuning parameter

```
caretRF$bestTune
```

```
##   mtry
## 1    2
```

Final Model

```
caretRF$finalModel
```

```
##
## Call:
##  randomForest(x = x, y = y, mtry = param$mtry, importance = TRUE)
##                Type of random forest: classification
##                      Number of trees: 500
## No. of variables tried at each split: 2
##
##         OOB estimate of  error rate: 20.24%
## Confusion matrix:
##    0  1 class.error
## 0 62 22   0.2619048
## 1 12 72   0.1428571
```

Figure 17: Confusion Matrix and Correct Rate for Best Random Forest Model

Confusion Matrix

```
table(predCaretRF, log.testCaret$starSize)
```

```
##
## predCaretRF  0  1
##           0 21  9
##           1 15 27
```

Correct Rate

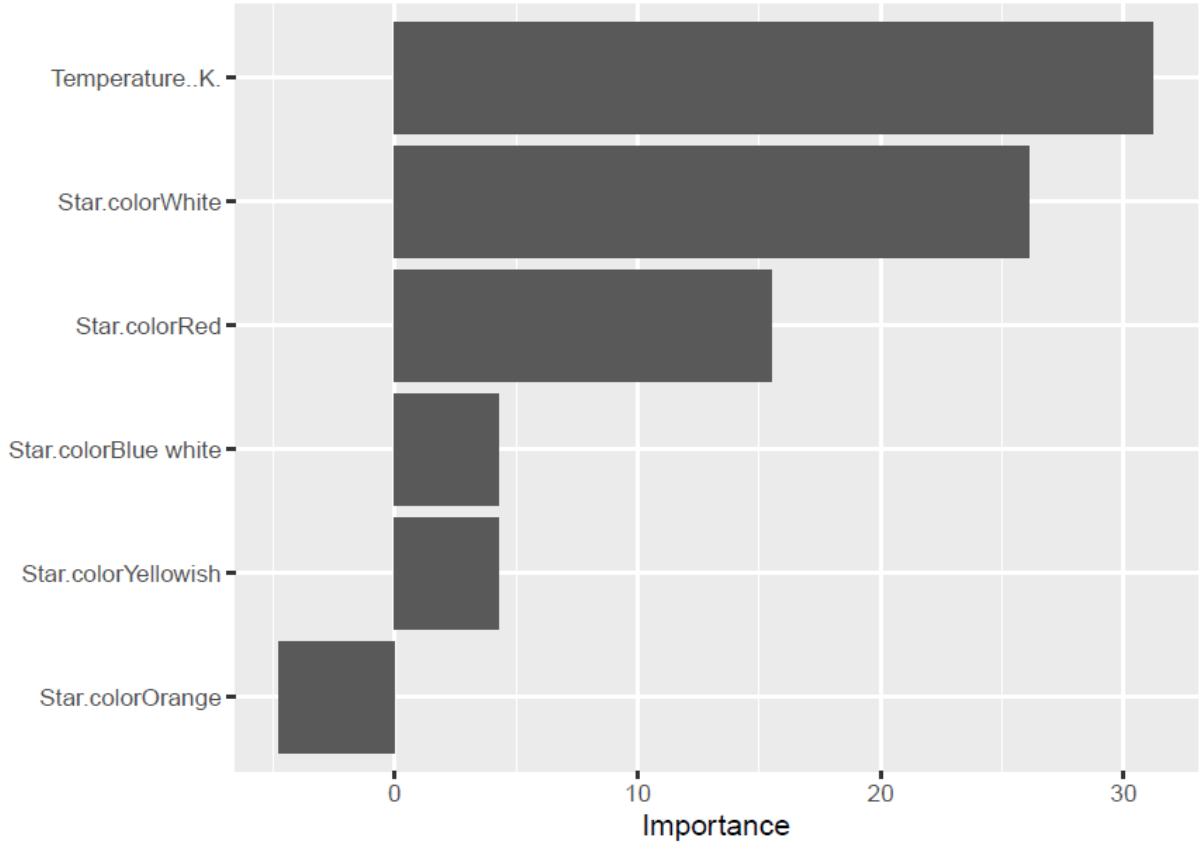
```
mean(predCaretRF==log.testCaret$starSize)
```

```
## [1] 0.6666667
```

Figure 18: Random Forest Variables by Importance

Variables by Importance

```
vip(caretRF$finalModel)
```



References

Baidya, D. (2019). *Star dataset to predict star types*. Kaggle. https://www.kaggle.com/datasets/deepu1109/star-dataset

Bates, J. (2021, February 7). *The stars within us*. National Science Foundation. https://www.nsf.gov/science-matters/stars-within-us

Mattson, B., Myers, J. D., & Ptak, A. (2013, August). *Brief history of timing analysis*. National Aeronautics and Space Administration: Goddard Space Flight Center. https://imagine.gsfc.nasa.gov/science/toolbox/timing_history.html

Star. (2025). Britannica Kids. https://kids.britannica.com/kids/article/star/353808

The Editors of Encyclopaedia Britannica. (2025, April 10). *Stellar classification*. Britannica. https://www.britannica.com/science/stellar-classification